Git y Shell Scripting en el día a día

Taller práctico: Sobrevive en el mundo dev

Asociación de estudiantes RoboTech

19 de marzo de 2026

Contenidos

- **Repaso Shell:** Por qué el ratón te está haciendo lento.
- **Comandos básicos**
- **Shell Scripting**
- **Repaso Git:** No más *v3_final_DEFINITIVA.zip*.
- **Comandos básicos Git:** Ramas y resolución de conflictos.
- **Ejercicio práctico**

¿Qué es la Shell?

- La **shell** es una capa de software que envuelve al Kernel. Tú escribes, ella traduce y el sistema ejecuta.
- **BASH** (Bourne Again Shell) es el estándar, pero hay más (Zsh, Fish).
- **Poder absoluto:** Puedes modificar archivos del sistema, gestionar procesos y redes con tres palabras.

Comandos básicos:

Los de supervivencia:

- `cd -`: Vuelve a la carpeta anterior (¡magia!).
- `ls -alth`: Lista todo por fecha.
- `mkdir -p`: Crea carpetas anidadas de golpe.

Los 'raros' pero útiles:

- `history`: ¿Qué escribí hace 10 minutos?
- `!!`: Ejecuta el último comando (útil para el `sudo !!`).
- `du -sh *`: ¿Qué carpeta me está llenando el disco?

El comando que lo encuentra todo

`grep -r 'error' .`

Busca la palabra 'error' en **todos** los archivos de la carpeta actual y subcarpetas. Es el buscador definitivo cuando el código falla.

Flags: ¿Cómo tunear tus comandos?

Un comando base (como `ls`) es funcional, pero los **flags** (opciones) son los que le dan superpoderes.

Tipos de banderas

- **Cortos:** Un solo guion y una letra. *Ej:* `-l`, `-h`.
- **Largos:** Dos guiones y una palabra completa. *Ej:* `--help`, `--all`.

Combinación de flags: En lugar de escribir `ls -l -a -h`, puedes agruparlos todos tras un solo guion:

```
ls -lah
```

El orden de los factores...

Casi siempre da igual el orden (`-lah` es igual a `-hal`), pero **¡CUIDADO!** si un flag necesita un argumento (como un nombre de archivo), ese debe ir al final.

Diccionario de flags: Las más usadas

Flag	Nombre	Efecto Mágico
`-r` / `-R`	*Recursive*	Entra en carpetas y aplica el comando a **todo** lo que haya dentro.
`-h`	*Human*	Traduce el formato para mejor comprensión.
`-f`	*Force*	Ignora avisos de seguridad.
`-a`	*All*	Muestra ficheros ocultos (que empiecen por .).
`-v`	*Verbose*	El comando explica todo lo que va haciendo.

Cada flag puede tener un uso distinto en cada comando, revisa el manual para poder trabajar bien.

El Combo de "Limpieza Total"

`rm -rf carpeta_vieja`

Recursivo + Fuerza. Borra la carpeta y todo su contenido sin pedir confirmación. **Úsalo con respeto.**

Expresiones regulares (Regex)

Regex es una forma de describir patrones de texto para buscar, filtrar o validar datos.

Patrones esenciales

```
.        # Cualquier caracter
^texto   # Empieza por "texto"
texto$   # Termina en "texto"
[0-9]    # Un digito
[a-z]+   # Una o mas letras minusculas
.*       # Cualquier cosa (0 o mas)
```

Expresiones regulares (Regex)

Ejemplos en consola

```
# Lineas que empiezan por "ERROR"
grep '^ERROR' app.log

# Correos simples (usuario@dominio.com)
grep -E '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}' contactos
    .txt

# IDs tipo DEV-123
grep -E '^DEV-[0-9]{3}$' ids.txt
```

AWK: filtrar y procesar texto

Lee texto línea por línea y te deja trabajar por columnas.

Idea clave

Formato base: `awk 'patrón { acción }' archivo`

```
# Mostrar la columna 1 y 3 de un CSV separado por coma
awk -F',' '{print $1, $3}' datos.csv
# Quedarte solo con procesos que usan más de 1% CPU
ps aux | awk '$3 > 1 {print $1, $2, $3, $11}'
# Contar líneas de un archivo (equivalente a wc -l)
awk 'END {print NR}' logs.txt
```

Truco útil

`-F` define el separador de columnas (coma, tab, etc.) y $1, $2... son las columnas. $NF es la última columna.

SED: edita texto

sed permite buscar y reemplazar texto sin abrir un editor.

Sintaxis base

```
sed 's/patrón/reemplazo/' archivo
```

```
# Reemplazar la primera coincidencia de cada línea
sed 's/error/ERROR/' app.log
# Reemplazar todas las coincidencias (flag g)
sed 's/error/ERROR/g' app.log
# Editar el archivo directamente (in-place)
sed -i 's/localhost/127.0.0.1/g' config.txt
# Mostrar solo líneas 10 a 20
sed -n '10,20p' app.log
```

Tip útil

El comando `s/.../.../g` significa *substitute*. Usa `-n` para evitar imprimir todo el archivo.

Shell Scripting:

Un script es simplemente un archivo de texto con una lista de órdenes. Úsalo para configurar entornos o robots de un solo golpe.

install_deps.sh

```
#!/bin/bash

echo "Instalando herramientas de RoboTech..."
sudo apt update
sudo apt install -y python3-pip git curl
pip install numpy matplotlib
echo "Entorno listo. A darle cana!"
```

Argumentos en un shell script

Cuando ejecutas un script como `./mi_script.sh dato1 dato2`, Bash guarda esos valores en variables especiales.

Variable	Qué representa
$0	Nombre/ruta del script ejecutado.
$1, $2, ...	Primer, segundo, ... argumento posicional.
$#	Número total de argumentos recibidos.
$@	Todos los argumentos (útil para recorrerlos).
$?	Código de salida del último comando (0 = OK).

```
#!/bin/bash
echo "Script: $0"
echo "Total args: $#"
echo "Primero: $1"
for arg in "$@"; do
  echo "-> $arg"
done
```

Variables

En Bash, las variables son cajas donde guardas texto o números. ¡Pero ojo con la sintaxis!

```
# DEFINIR (Sin espacios alrededor del '=')
NOMBRE="R2D2"
VERSION=2.0

# USAR (Siempre con el simbolo '$')
echo "Hola, soy $NOMBRE y mi version es $VERSION"

# GUARDAR RESULTADOS DE COMANDOS
ARCHIVOS=$(ls | wc -l)
echo "Tengo $ARCHIVOS ficheros en esta carpeta"
```

La regla de oro: Cuidado con los espacios

`NOMBRE = 'R2D2'` → **ERROR.**

Para la Shell, eso significaría: Ejecuta el programa `NOMBRE` con los argumentos `'='` y `'R2D2'`. **Pégalo al igual.**

Condicionales: Toma de decisiones

El `if` evalúa si una prueba es cierta. Se cierra con `fi` (if al revés).

Estructura IF / ELSE

```
if [ "$VARIABLE" == "RoboTech" ]; then
    echo "Acceso concedido, master."
else
    echo "Identificate, humano."
fi
```

Pruebas comunes (Flags de utilidad)

- `[ -d` *DIR*`]` : ¿Existe esta carpeta?
- `[ -f` *FILE*`]` : ¿Existe este archivo?
- `[ $A -gt $B ]`: ¿Es A mayor que B? (*También* `-lt` *y* `-eq`).

Loops

¿Tienes que procesar 500 fotos de un sensor? No lo hagas a mano. Los bloques usan do (haz) y done (hecho).

Bucle FOR (Lista fija):

```
for i in {1..5}; do
    echo 'Carga:\"   $i'
done
```

Bucle WHILE (Condicion):

```
while [ $N -lt 10 ]; do
    echo "N vale $N"
    N=$((N+1))
done
```

El poder del comodin (*)

```
# Procesar todas las fotos
for img in *.jpg; do
    mv "$img" "${img%.jpg}_backup.jpg"
done
```

El poder del .bashrc

Un **alias** es una forma de definir un comando con otro nombre:

Ejemplos para tu /.bashrc

```
# Alias para no morir escribiendo
alias gs='git status'
alias gca='git commit -a -m'
alias rmaster='git checkout main && git pull'
alias cls='clear'
alias sl='ls'
```

`source ~/.bashrc` para activar los cambios al instante.

SSH

Imagínate que tu robot está al otro lado de la habitación (o del mundo). ¿Le conectas un monitor? **No.**

- **SSH (Secure Shell):** Protocolo para acceder de forma segura a otra máquina.
- **SCP:** Para mandar archivos como si fuera un cp pero por red.

```
# Entrar al robot (Raspberry Pi, Jetson, etc)
ssh robotech@192.168.1.XXX

# Traerte los logs a tu PC local
scp robotech@192.168.1.XXX:~/logs/sensor.txt ./
```

¿Qué es Git? (Y por qué te salva la carrera)

Git es un **Sistema de Control de Versiones**. Imaginalo como el sistema de "Guardar Partida" de un videojuego, pero para tu código.

- Snapshot, no copia: Git saca una "foto" de todo tu proyecto en un momento dado (*commit*).
- Historial Infinito: Puedes volver a como estaba el código hace 5 minutos, 3 días o 2 años.
- Integridad: Es imposible que un archivo se corrompa sin que Git se dé cuenta.

El caos que vamos a matar

Git elimina para siempre los archivos tipo:
`proyecto_FINAL.py`, `proyecto_V2_ESTE_SI.zip`...
Con Git solo tienes un archivo, pero con mil versiones ocultas dentro.

Git $\neq$ GitHub / GitLab

Es el error típico de principiante. No son lo mismo, aunque trabajen juntos.

Git (La Herramienta)

- Es el **motor** que instalas en tu PC.
- Funciona **localmente**.
- No necesita internet.
- Es donde creas las versiones.

GitHub/GitLab (La Red)

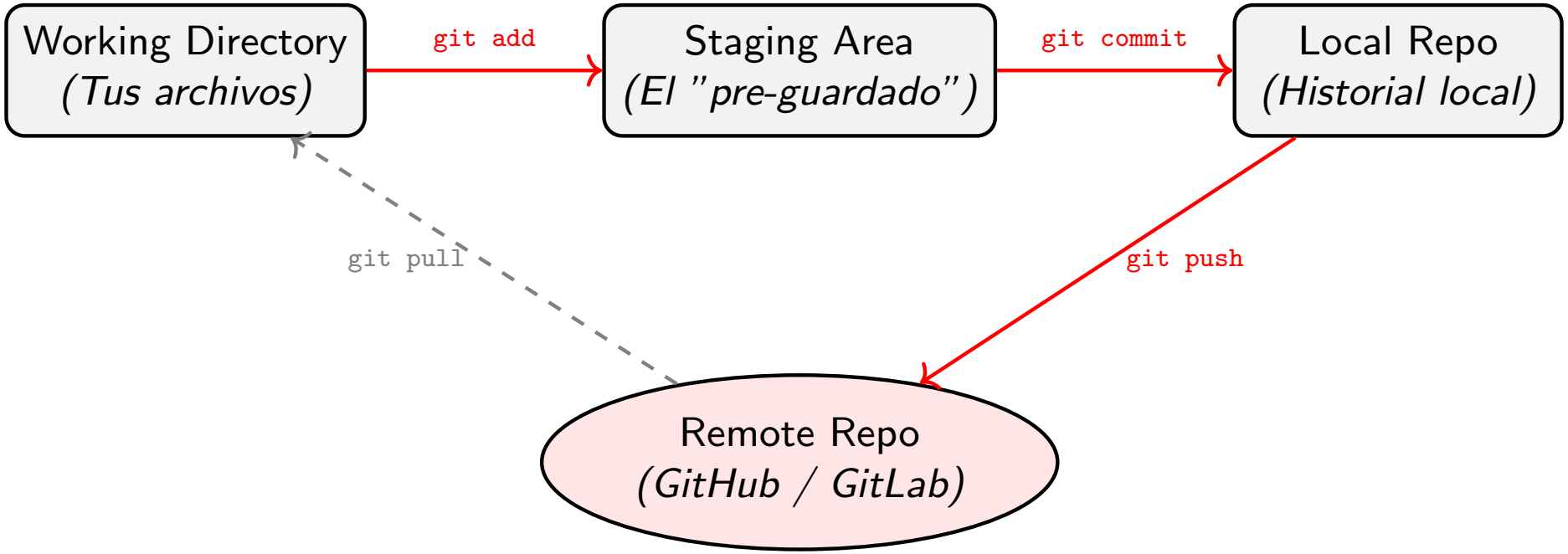
- Es la **nube** (el servidor).
- Donde 'aparcas' tu código para que otros lo vean.
- Permite colaborar en equipo.

Analogía

Git es como tu consola (donde juegas y guardas). **GitHub** es como Twitch (donde subes lo que haces para que el mundo lo vea y comente).

Git Workflow

Git no guarda las cosas directamente. Tiene un sistema de 3 capas locales antes de llegar a la nube (GitHub).



El "Limbo" del Staging

¿Por qué existe el Staging? Porque a veces haces cambios en 5 archivos pero solo quieres guardar 2. El `git add` te permite elegir qué "foto" vas a sacar, sin mezclar basura o experimentos que no funcionan.

Comandos de flujo

1. `git add <archivo>`: Metes el archivo en la caja. Está listo para la foto.
2. `git commit -m "mensaje"`: Cierras la caja y le pones una etiqueta. Ya tienes un punto de restauración.
3. `git push`: Subes todas tus cajas etiquetadas al servidor.

- git commit -am "mensaje"
  Hace el `add` (de archivos modificados) y el `commit` de un solo tiro.

Dato clave

Si no haces `push`, tus compañeros no ven nada. Si se te rompe el PC, pierdes todo lo que no esté en el `Remote Repo`.

Git

Git no es solo un backup. Es una 'máquina' del tiempo.

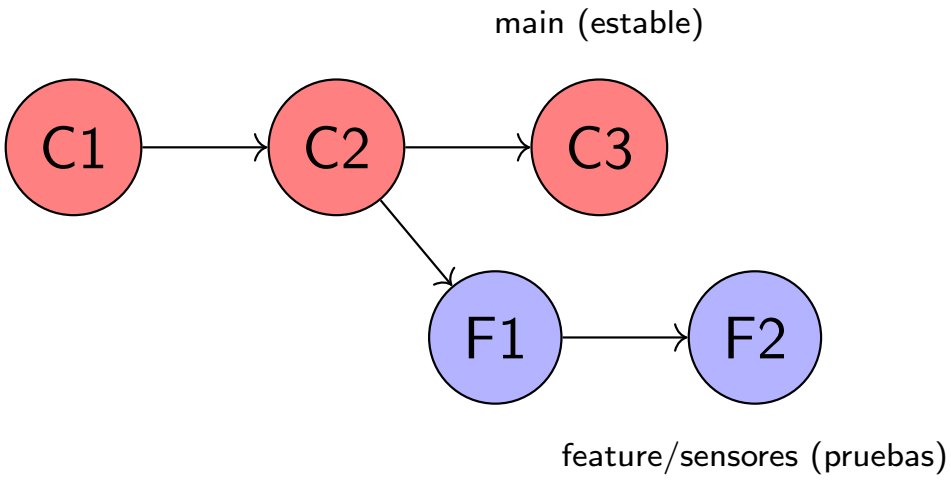
- **Grafo dirigido:** Cada commit sabe quién es su padre.
- **Integridad:** SHA-1 asegura que nadie cambie el código por detrás.

El error de los 100 millones

Hay empresas que han perdido meses de trabajo por no usar Git o por borrar el repositorio sin tener remotos.

Ramas: Trabajo en paralelo

No toques el `main` si no estás seguro. Crea una rama.



¿Por qué?

Si la rama `feature` falla, la borras y el código estable sigue intacto.

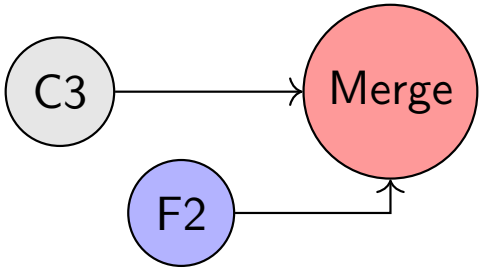
Git Merge

Integrar la rama `feature` en `main`.

- **Fast-forward:** Si no hay cambios en `main`, es instantáneo.
- **3-way Merge:** Si ambos han cambiado, Git intenta unirlos.

!) Conflicto

Si dos personas modifican la misma línea, se genera un conflicto. Se selecciona que cambios queremos integrar y solucionado.



Comandos de Supervivencia

Comando	Caso de uso
`git stash`	"Me tengo que ir a comer pero no quiero hacer commit de este desastre".
`git checkout -- file`	"He borrado algo sin querer y quiero volver atrás".
`git log --graph`	"Quiero ver el árbol de commits como un artista".
`git reset --hard`	"Borra todo rastro de los últimos 10 minutos, me he equivocado de planeta".

Reto Final: Operación Rover Marte

Instrucciones para el ejercicio

1. Haced grupos de 3 personas.
2. Haced fork del siguiente repo (cada uno en su pc): https://github.com/RoboTech-URJC/shell_workshop_rover
3. En el fichero '.pdf' encontraréis el enunciado e indicaciones.

¡Domina la terminal!

Dudas, insultos o café en:
asociacion.robotech@urjc.es